

AUTOMATED GOLD TRADING SYSTEM

# xAuby

A Plain-Language Guide — for people getting started  
with the gold trading bot

---

Compiled and rewritten from every project document  
Rewritten for clarity, with technical jargon minimized  
for beginners

Reflects the system's latest state as of July 2026  
Prepared for education and internal use only

# Table of Contents

## Preface and Current System Status

### Part 1 — Meet xAuby

- 1.1 What is xAuby
- 1.2 The Thinking Behind the Design
- 1.3 How the System Is Configured Right Now
- 1.4 What This Bot Does and Doesn't Do

### Part 2 — Getting Started

- 2.1 Installation
- 2.2 Setting Up Your OKX API Key
- 2.3 Always Run Simulation First
- 2.4 Common Commands
- 2.5 Checklist Before Going Live

### Part 3 — How the System Works Behind the Scenes

- 3.1 A Simple Overview of the Pieces
- 3.2 What Happens on Every Cycle
- 3.3 How Multiple Trading Pairs Stay Separate
- 3.4 Ground Rules the System Always Follows
- 3.5 Strategies and Charts Must Travel Together
- 3.6 Feature Switches

### Part 4 — Configuration

- 4.1 The Three Main Configuration Files
- 4.2 Why Configuration Is Split Into Three Groups
- 4.3 Exchange Setup (OKX via CCXT)
- 4.4 The List of Active Trading Pairs
- 4.5 Environment Variables Worth Knowing
- 4.6 The Built-in Quick Config Menu

### Part 5 — Strategies and the Market Regime Detector

- 5.1 Every Strategy in the System
- 5.2 CDC Action Zone — the Strategy Actually Trading Live
- 5.3 What Is the Market Regime Detector
- 5.4 A Statistical Second Opinion
- 5.5 How Accurate Is the Regime Detector, Really

- 5.6 Automatically Picking a Strategy for the Market
- 5.7 How Strategy Changes Get Decided With Real Data

## **Part 6 — Risk Management**

- 6.1 How the Bot Sizes a Position
- 6.2 Stop-Loss, Trailing, and Breakeven
- 6.3 Taking Partial Profit Early
- 6.4 The Portfolio-Wide Loss Switch
- 6.5 Short Trading and Its Safety Conditions
- 6.6 Keeping Risk Settings Consistent

## **Part 7 — Backtesting and Research**

- 7.1 Where Backtest Data Comes From
- 7.2 Backtest Commands
- 7.3 Regime-Aware Replay
- 7.4 Validating Correctness After a Restart

## **Part 8 — User Interfaces**

- 8.1 The Terminal Dashboard (TUI)
- 8.2 The Read-Only Mobile WebUI
- 8.3 Design Philosophy

## **Part 9 — Telegram**

- 9.1 Setting Up the Telegram Bot
- 9.2 All Commands
- 9.3 Automatic Alerts
- 9.4 Confirm-Before-Buy Mode and the Emergency Pause

## **Part 10 — Multiple Accounts and Exchanges**

- 10.1 Running Several Bots on One Machine
- 10.2 Switching to a Different Exchange
- 10.3 OKX Perpetuals and Short-Trading Safety

## **Part 11 — Operations**

- 11.1 Deploying on a VPS
- 11.2 Update and Restart Commands
- 11.3 Diagnosing a Slow VPS
- 11.4 The Full Safety Checklist

## **Appendix**

- A. Glossary
- B. Project Layout
- C. Command Reference
- D. Risk Disclaimer

## Preface and Current System Status

This book gathers every document in the xAuby project into a single reading order, written in a plain, conversational style, with as much technical jargon stripped out as possible while keeping every important detail. It's written for someone who has just started using this bot and isn't yet comfortable with trading or programming terminology.

**Where to start reading:** if you just want to get up and running quickly, read in this order first — **Part 1** → **Part 2** → **Part 6** → **Part 11** (meet the system, install it, understand the risk, and the checklist before using real money). **Part 3, sections 5.3-5.7, and 8.3** go deeper into how things work internally — feel free to skip them on a first read and come back later when you're curious.

**A note on the older docs:** some of the project's older documentation still describes an earlier version of the system (back when it traded on a different exchange with two pairs running at once). That's out of date now. This book is based on the system's real configuration files, so everything written here matches what the system actually does today, not an older version that has since changed.

### The System's Real Current Setup (Quick Summary)

| Item                                             | Actual Value                                                                      |
|--------------------------------------------------|-----------------------------------------------------------------------------------|
| Exchange used                                    | OKX (through a connector called CCXT that lets the bot talk to several exchanges) |
| Pair actually trading live                       | <code>XAUUSD</code> (gold) — just this one pair                                   |
| Strategy in use                                  | <code>cdc_action_zone</code> — trades both up moves (long) and down moves (short) |
| Timeframe watched                                | Main chart every 4 hours, daily chart to confirm the bigger direction             |
| Leverage                                         | 1x (no borrowed money added to the risk)                                          |
| Risk per trade                                   | Risks 2% of the portfolio on each entry                                           |
| Maximum position size                            | No more than 25% of the portfolio per trade                                       |
| Maximum daily loss                               | If losses reach 6% in a day, the bot stops opening new trades                     |
| Max trades open at once                          | Just 1                                                                            |
| Stop-loss                                        | Disabled — this strategy exits purely on a signal flip, not a fixed stop price    |
| Early partial profit-taking                      | Sells off half the position once profit hits 12%                                  |
| Automatic strategy switching by market condition | Turned off — the bot uses one fixed strategy, it doesn't switch on its own        |

These numbers can change whenever the operator updates the configuration. Always check the real configuration files before relying on any of this for an important decision.

## 1.1 What is xAuby

**xAuby** is an automated gold trading program. It runs around the clock on **OKX** without anyone needing to watch a screen. The bot continuously pulls in the latest gold price, analyzes it against the strategy it's configured with, and decides on its own whether to buy or sell — complete with risk protection and Telegram alerts so the owner can check in from anywhere, anytime.

The system can run several trading pairs at once (right now only one — gold — is actually turned on), and can be controlled through several channels: terminal commands, a terminal dashboard screen, and Telegram.

## 1.2 The Thinking Behind the Design

---

The principles the system has followed from the start, in plain terms:

- **Everything is configured through files, not code changes** — want to change the trading pair, the strategy, or any setting? Edit a configuration file. You don't need to be a programmer.
- **Strategies plug in and out like modules** — the core engine isn't tied to any one strategy. New strategies can be added without touching the rest of the system.
- **Configuration is split into three clear groups** (details in Part 4) so that backtest results always match what happens in live trading.
- **Safe by default** — the system always starts in simulation mode. Switching to real money requires setting several confirmations at once (see section 2.5), which prevents accidentally trading real funds.
- **Each trading pair is fully isolated** — if several pairs are running, a problem with one won't affect another. The trading capital itself is intentionally shared across all pairs.
- **Features can be switched off instantly** — every major feature can be turned back off with a single flip of a setting, no code changes or reinstalling required.

## 1.3 How the System Is Configured Right Now

---

| Pair   | Mode | Strategy        | Main chart | Confirm chart | Sides allowed |
|--------|------|-----------------|------------|---------------|---------------|
| XAUUSD | Live | cdc_action_zone | 4 hours    | Daily         | Long + Short  |

The bot trades **OKX Perpetual Swaps** (contracts with no expiry date) at **1x leverage** — meaning no borrowed money is added to the risk at all. It also checks the funding rate (the interest rate paid between long and short traders) to make sure it isn't abnormally high, and always keeps a safe distance from the liquidation price. Backtesting uses a gold-tracking token instead of the live gold contract, since it has a much longer price history that closely mirrors real gold.

The live strategy can open **both long and short at once**, in a **stop-and-reverse** style — in plain terms: if the bot is holding a long position and the signal flips to bearish, it closes the long and opens a short immediately, in the same beat, with no waiting.

**How orders get sent:** when entering a trade, the bot first places a limit order at a set price. If that order sits unfilled too long, the bot switches to a market order instead, so it doesn't miss the move. When exiting in an emergency (a sudden signal flip, or a forced close), the bot always uses a market order for speed. Otherwise it uses the set-price order.

The current gold strategy has **no traditional stop-loss** — every exit is driven purely by the signal changing, and position size is calculated as a fixed fraction of capital rather than based on stop distance. It also takes an early partial profit: once profit reaches 12%, the bot sells 50% of the position, then lets the rest ride on the normal exit signal.

**An important safeguard:** if a live trading pair has automatic strategy-switching turned on but hasn't been formally approved by the operator, the bot will **automatically force that pair back into simulation mode** — this keeps unapproved features from ever touching real money. And whenever the market is judged to be in a dangerous zone (like a panic sell-off), the bot won't open new trades, keeps existing protection in place, tightens its safety margins, and can force-close a position if the dangerous conditions drag on too long.

## 1.4 What This Bot Does and Doesn't Do

---

### What it does

- Analyzes trading signals from price charts, based on the configured settings
- Opens and closes real positions on the exchange, or simulated ones for testing
- Keeps basic risk protection active at all times
- Logs every event so you can review it later on-screen, when troubleshooting, or via Telegram

### What it doesn't do

- It does not guarantee any profit
- It does not manage funds sitting in any OKX wallet other than the one configured for trading
- It does not automatically make a strategy written by someone else safe, unless the code-scanning feature is turned on (see Part 3)
- It does not replace your own review of risk, leverage, fees, and the exchange's rules

#### Warnings for new users:

1. Right now the system is configured for exactly one live market: gold ( **XAUUSD** ) on OKX.
2. You must get simulation mode working successfully before ever considering real money.
3. Trading with real money requires three conditions set at the same time: a command-line flag, a confirmation environment variable, and that specific pair marked as live in its configuration.

4. The bot only reads the account actually configured for trading on OKX — not every wallet in your OKX account. If the dashboard shows a balance of 0.00, check that your funds are sitting in the trading account, and that the API key has permission to read/trade that account.
5. **Never enable withdrawal permission on the API key.** The bot only ever needs permission to read and to trade — it never needs to withdraw funds.
6. Only run one instance of the bot per data folder. If you need multiple accounts, you must separate the data folders (see Part 10).

## 2.1 Installation

What you need first: a machine with Python 3.12 or newer, an exchange API key, and a Telegram bot (optional, only if you want alerts).

```
git clone https://github.com/iisara555/xAuby.git
cd xAuby

python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install -e .

cp .env.example .env
python run_xauby.py --simulate
python health_check.py
```

## 2.2 Setting Up Your OKX API Key

Before going live, open the `.env` file and fill in your OKX connection details:

```
OKX_API_KEY=...
OKX_API_SECRET=...
OKX_API_PASSPHRASE=...
LIVE_TRADING=true
```

OKX requires all three: an API Key, a Secret, and a Passphrase (an extra API password). **Never upload your `.env` file to Git or anywhere public** — it's the key to your trading account.

Before trading for real, verify the pair and exchange connection:

```
python scripts/evaluate_okx_xau_migration.py --config configs/okx_xau_paper.yaml --whitelist configs/okx_xau_paper.yaml
python health_check.py
```

## 2.3 Always Run Simulation First

The system always starts in **simulation mode** by default. No matter what gets misconfigured, the system leans toward the safe option. Switching to live trading requires an explicit set of steps — see section 2.5.

Other ways to launch the bot:



| Command                                              | What it does                                              |
|------------------------------------------------------|-----------------------------------------------------------|
| <code>python run_xauby.py --simulate / --live</code> | Starts the trading engine directly (simulation / live)    |
| <code>python launcher.py</code>                      | Opens an interactive menu (engine plus dashboard options) |
| <code>xauby / xauby --sim / xauby --live</code>      | Shortcut commands, available once installed               |
| <code>xauby restart [--live]</code>                  | Restarts with a safety check first                        |
| <code>xauby update</code>                            | Pulls the latest code from GitHub, then restarts          |
| <code>python -m xauby.ui.textual_tui.app</code>      | Opens the dashboard screen on its own                     |
| <code>python health_check.py</code>                  | Runs a one-time health check                              |

## 2.4 Common Commands

Recommended command to go live, after simulation and a health check have both passed:

```
env -u DEFAULT_SYMBOL XAUBY_DEFAULT_SYMBOL=XAUUSD XAUBY_FOCUS_SYMBOL=XAUUSD \
SIMULATE_ONLY=false BOT_READ_ONLY=false \
./venv/bin/python run_xauby.py --live --pair XAUUSD
```

To open a read-only dashboard from your phone or browser:

```
./scripts/start_webui.sh
```

Commands you'll use often while operating the bot:

```
xauby restart          # Restart the engine, with a safety check first
xauby restart --live   # Force a checked restart specifically for live mode
xauby update           # Pull the latest code from GitHub, then restart
```

The checked restart path will accept positions the bot is already tracking, but will refuse to proceed if it finds an unknown balance or an untracked order — this stops the bot from touching anything it didn't open itself.

## 2.5 Checklist Before Going Live

Going live requires every one of these conditions at the same time. Miss even one, and the bot won't send a real order:

1. Pass a live flag when you run the command
2. Set the live-trading confirmation environment variable
3. The YAML setting must not be set to simulation-only
4. That specific pair must be marked as live in the pair list file

| Mode               | How to turn it on                       | Behavior                                                          |
|--------------------|-----------------------------------------|-------------------------------------------------------------------|
| Simulation         | Default, or set per pair                | No real orders — like a practice run                              |
| Live               | Requires all 4 conditions above         | Sends real orders to the exchange                                 |
| Read-only          | A specific setting                      | Processes signals only, never sends any order                     |
| Confirm-before-buy | A semi-automatic mode setting           | Waits for a Telegram confirmation before every buy                |
| Pause via Telegram | Type <code>/pause</code> , then confirm | Blocks all new buys system-wide; <code>/resume</code> to continue |

The full, detailed safety checklist lives in section **11.4** at the end of this book — read it in full before ever putting real money in.

# 3.1 A Simple Overview of the Pieces

This part explains how the bot works under the hood — useful if you want a deeper understanding. If you just want to operate the bot, feel free to skip ahead to Part 4 (Configuration) and come back later.

xAuby splits its work across 5 cooperating pieces:

| Piece               | What it's responsible for                                                                 |
|---------------------|-------------------------------------------------------------------------------------------|
| Human control layer | Terminal commands, the dashboard screen, and Telegram — where a person gives instructions |
| Core engine         | Makes the actual trading decisions, manages risk, and sends orders                        |
| Strategy layer      | Calculates trading signals and chart data                                                 |
| Data layer          | Pulls prices from the exchange and stores data                                            |
| Event logging       | Records everything that happens, for later review and for driving the screens             |

The rough flow: a person starts the bot → the bot loads its configured trading pairs → each pair is processed separately → (if enabled) a strategy is picked automatically based on market conditions → trading signals are calculated → risk conditions are checked → a real or simulated order is sent → everything gets logged to the database → the screens update with the latest result.

## 3.2 What Happens on Every Cycle

The bot works in cycles, called "ticks" (for example, checking in every 60 seconds). On every cycle, it goes through this sequence for each active trading pair:

### Steps for each trading pair

1. Pull the latest price candles from the exchange
2. Check whether the fast price feed (WebSocket) is still fresh — if it's too stale, fall back to a backup price source instead
3. Check whether a position is currently open
4. Load the strategy and capital settings for that pair
5. (If enabled) analyze market conditions and automatically pick the right strategy
6. If market conditions are judged dangerous, skip opening new trades and tighten protection
7. Otherwise, let the strategy calculate its trading signal as normal
8. Check for abnormal news/market conditions before allowing any buy
9. Act on the signal: open a new trade, close one, or hold while managing partial profit-taking and the trailing stop
10. Log everything and update the screens

## When opening a new trade

1. Check whether today's trade count or loss limit has already been hit
2. Check that the order size isn't too small, or the price too far from what was intended
3. Calculate position size from the available capital and the configured risk percentage
4. Send a real or simulated order, depending on that pair's mode
5. Set risk protection (if that strategy uses one)
6. Send a Telegram alert confirming the buy/sell went through

## When closing a trade

Several things can trigger an exit: a sell signal from the strategy, an early partial profit-take hitting its target, a stop-loss that's been confirmed repeatedly, detecting that a stop order already filled on the exchange, a forced close after too long in a dangerous market condition, or a maximum holding time being reached. If a sell fails, the system tries to re-establish protection and sends a critical Telegram alert if that also fails.

## Taking partial profit early

This feature only runs **after** the system has already confirmed there's no full-exit signal in the same cycle (a full exit always wins if both happen at once). Once the current price is better than the entry price by the configured percentage, the bot sells part of the position, resizes any protection to match what's left, and lets the remainder ride on the normal exit signal.

## Safely handing off between strategies

Think of it like changing drivers mid-trip — if the system switches strategies while a position is open, the original strategy keeps managing that position until it closes. The new strategy won't touch it until there's nothing left open. This avoids confusion over who's responsible for which trade.

## When the market is in a dangerous zone

Once market conditions are classified as dangerous, the bot will: block all new trades, keep the existing protection on any open position, tighten its safety margins, wait for conditions to normalize for a while before trading again, and can force-close a position if the dangerous conditions persist too long.

## Short trading

Strategies that support short trades send separate "open short" / "close short" signals rather than reusing the normal buy/sell signals. The stop-loss for a short sits above the entry price (the mirror image of a long). The bot always double-checks its recorded position against the real position on the exchange, and will stop trading immediately if they don't match — this prevents the bot from getting confused about what it's actually holding.

### 3.3 How Multiple Trading Pairs Stay Separate

---

The system loads its list of trading pairs from a configuration file. If the "pair list is the single source of truth" setting is on, that file governs everything, and if "auto reload" is also on, the system checks that file periodically and picks up new pairs without needing a restart.

Each active trading pair keeps its own fully separate data, including:

- Its latest price
- The strategy currently in use (and a handoff target, if one is in progress)
- Its latest market condition
- Its mode — simulation or live
- Any pending confirm-before-buy state
- Its latest signal data, for the screens and Telegram

Trading pairs are isolated at the strategy and processing level, but the trading capital and overall risk conditions (like the daily maximum loss) are intentionally shared across all of them.

### 3.4 Ground Rules the System Always Follows

---

| Rule                                                   | Why                                                                             |
|--------------------------------------------------------|---------------------------------------------------------------------------------|
| Event logging never depends on the core engine         | So you can still review history even if the engine itself has a problem         |
| The dashboard screen only ever reads data              | The screen must never block or accidentally change trading data                 |
| Secrets live only in the <code>.env</code> file        | The main configuration file can be shared safely, since it never holds any keys |
| Only one instance runs per data folder                 | Prevents accidental duplicate trading                                           |
| A strategy and its chart must always travel together   | What you see on the chart must match what the strategy is actually doing        |
| The core engine is never tied to one specific strategy | All trading logic lives in configuration/plugins only                           |

### 3.5 Strategies and Charts Must Travel Together

---

This is the most important rule for extending the system: **every new strategy must ship with a matching chart display**. Add a new strategy without a compatible chart, and the screen will show something that doesn't match what the strategy is actually doing — leaving the user with a wrong impression of what the bot is up to.

A strategy must be a "pure analyst" only — it must never send orders itself, touch the database, or call Telegram directly. Its only job is to analyze and hand back a signal. The core engine is the one that actually

decides what to do with that signal.

## Checklist for adding a new strategy (for developers)

1. Write the new strategy and register it with the system
2. Write a matching chart display (or reuse a compatible one)
3. Add the strategy-to-chart mapping in the configuration file
4. Write tests covering the strategy, its chart display, and the on-screen legend

Some strategies in the system are tagged "research only" — the system will **automatically block** that entire group from ever running on a live pair. They can only be used in simulation or backtesting.

## 3.6 Feature Switches

Every major feature in the system has an on/off switch in the configuration file. To disable any feature temporarily or permanently, just flip its switch off — no code changes needed.

| Feature                                          | What it does                                                                                                           |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Pair list as single source of truth              | Makes the pair list file the one source of pairs/strategies, with validation before startup                            |
| Plugin-based chart rendering                     | Charts compute their lines through a plugin system instead of hard-coded logic                                         |
| Auto-sync pair list                              | Keeps the active pair list in sync with the configuration file automatically                                           |
| Per-pair execution mode                          | Lets each pair choose simulation or live independently                                                                 |
| Simulated broker with real ledger                | Simulated orders get their own proper account ledger, just like a real one                                             |
| Automatic strategy switching by market condition | The master switch for this feature (still needs to be enabled per pair too — see 5.6)                                  |
| Market-condition confidence filter               | Suppresses a market-condition switch when confidence is below the threshold                                            |
| Statistical regime cross-check                   | Turns on the parallel statistical checker (advisory only — see section 5.4)                                            |
| Strategy code scanning before use                | Rejects (not just warns about) a strategy whose code looks untrustworthy — useful when using plugins from other people |

## 4.1 The Three Main Configuration Files

| File                             | Safe to commit to Git? | What it holds                                          |
|----------------------------------|------------------------|--------------------------------------------------------|
| <code>.env</code>                | No (contains secrets)  | API keys, the live-trading flag, Telegram settings     |
| <code>.env.example</code>        | Yes                    | A blank template for new installs                      |
| <code>bot_config.yaml</code>     | Yes                    | Engine, strategy, portfolio, and notification settings |
| <code>coin_whitelist.json</code> | Yes                    | Active trading pairs, strategy per pair, mode          |

Files the bot creates while running (database, logs, simulated balances) are specific to that machine — never commit them to Git, and don't copy them straight to another machine unless you actually intend to move the whole running system.

## 4.2 Why Configuration Is Split Into Three Groups

All settings are divided into three clearly separated groups. Putting a value in the wrong group is the most common source of bugs, because backtesting and live trading must read values from the same place — get it wrong, and backtest results will stop matching live results.

| Group                     | Owns                                                                               | Must not own                    |
|---------------------------|------------------------------------------------------------------------------------|---------------------------------|
| <b>Engine settings</b>    | Exchange connection, logging, notifications, overall risk limits, feature switches | Strategy signal/exit parameters |
| <b>Strategy settings</b>  | Indicator values, stop-loss, trailing, breakeven, timeframe, signal/exit logic     | Secrets or position sizing      |
| <b>Portfolio settings</b> | Position sizing, capital allocation                                                | Indicator/signal rules          |

**The rule to remember:** anything that can change a backtest result belongs in the strategy or portfolio group only — never in the engine group.

## 4.3 Exchange Setup (OKX via CCXT)

The `exchange:` block in the main configuration file is the single source of truth for which exchange to trade on. Every part of the system (the engine, backtesting, health checks) reads this same value, so simulation and live trading always stay consistent.

```
exchange:
  provider: ccxt
  ccxt_id: okx
  name: okx
  quote_asset: USDT
  settle_asset: USDT
  fee_pct: 0.0005
  market_type: swap
  margin_mode: isolated
  position_mode: one_way
  api_key_env: OKX_API_KEY
  api_secret_env: OKX_API_SECRET
  base_url_env: OKX_BASE_URL
```

- **Connection details:** read from the environment variables named in this setting. OKX needs all three: API Key, Secret, and Passphrase.
- **Trading fee:** the system checks several places in order — a pair-specific setting wins for that pair specifically, if one is set.
- CCXT is the main connector actually used for OKX right now.

## 4.4 The List of Active Trading Pairs

---

```
{
  "version": 1,
  "quote_asset": "USDT",
  "assets": [
    {
      "symbol": "XAU",
      "enabled": true,
      "primary_timeframe": "4h",
      "confirm_timeframe": "1d",
      "strategy": "cdc_action_zone",
      "mode": "live",
      "allowed_sides": ["long", "short"],
      "leverage": 1.0,
      "short_live_enabled": true,
      "regime_router_enabled": false,
      "regime_router_live_confirmed": false,
      "sim_fee_pct": 0.05
    }
  ]
}
```



| Field                                     | Possible values                   | Notes                                                              |
|-------------------------------------------|-----------------------------------|--------------------------------------------------------------------|
| <code>symbol</code>                       | Base asset, e.g. <code>XAU</code> | The system automatically appends the quote currency                |
| <code>enabled</code>                      | true/false                        | Disabled pairs are skipped entirely                                |
| <code>primary_timeframe</code>            | e.g. 1h, 4h                       | Used for live trading, backtesting, and charts                     |
| <code>strategy</code>                     | Strategy name                     | Required when "pair list as single source of truth" is enabled     |
| <code>mode</code>                         | Simulation / Live                 | Requires per-pair execution mode to be enabled first               |
| <code>regime_router_enabled</code>        | true/false                        | The per-pair switch for automatic strategy selection               |
| <code>regime_router_live_confirmed</code> | true/false                        | Must be on before a live pair can use automatic strategy selection |
| <code>sim_fee_pct</code>                  | Number                            | Simulated trading fee, as a percentage                             |

## 4.5 Environment Variables Worth Knowing

| Variable                                                       | Default           | Description                                                                                          |
|----------------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------|
| <code>OKX_API_KEY / OKX_API_SECRET / OKX_API_PASSPHRASE</code> | -                 | OKX connection details                                                                               |
| <code>LIVE_TRADING</code>                                      | Off               | An extra gate that must be on to allow real orders                                                   |
| <code>BOT_READ_ONLY</code>                                     | Off               | Skips sending any order at all                                                                       |
| <code>TELEGRAM_ENABLED</code>                                  | Off               | The master switch for Telegram alerts                                                                |
| <code>TELEGRAM_BOT_TOKEN</code>                                | -                 | Your bot's token from @BotFather                                                                     |
| <code>TELEGRAM_CHAT_ID</code>                                  | -                 | The chat authorized to control the bot                                                               |
| <code>XAUBY_HOME</code>                                        | <code>core</code> | The folder holding the bot's data — relocate it to run multiple instances side by side (see Part 10) |

## 4.6 The Built-in Quick Config Menu

Open the launcher menu and choose **Quick Configuration** for a point-and-click settings screen — no manual YAML editing required. Navigate with arrow keys, Enter, or the mouse; secrets are always hidden. Editing through this menu never destroys existing comments or structure in the configuration file.

| Menu group                   | What you can edit                                                    |
|------------------------------|----------------------------------------------------------------------|
| Trading & Risk               | Mode, risk % per pair, stop-loss distance, strategy                  |
| Overall Trading              | Max open trades, timeframe, minimum order size                       |
| Risk Guards                  | Portfolio-wide stop switch, maximum daily loss %                     |
| Portfolio                    | Starting capital, allocation % per pair                              |
| Automatic Strategy Selection | On/off, thresholds, market-condition-to-strategy mapping             |
| Telegram Alerts              | Connection details/testing, daily/weekly report schedule             |
| Exchange                     | A guided wizard to switch exchanges step by step, connection testing |

## 5.1 Every Strategy in the System

The system ships with 15 strategies total. Some are used in live trading; others are research-only and can't yet be traded with real money:

| Strategy                             | Style                                                                   |
|--------------------------------------|-------------------------------------------------------------------------|
| <code>cdc_action_zone</code> ★ Live  | Trend-following using two moving averages, auto-reverses direction      |
| <code>supertrend_ema200</code>       | Trend-following using a SuperTrend line and a 200-period moving average |
| <code>bbkc_squeeze</code>            | Waits for price to squeeze tight, then trades the breakout              |
| <code>bbrsi_mean_reversion</code>    | Buys dips and sells rallies (short-term mean reversion)                 |
| <code>btc_ema_pullback</code>        | Follows the trend, then waits for a pullback to enter                   |
| <code>ict_lite_strategy</code>       | Based on liquidity-hunt concepts, then waits for a reversal             |
| <code>rsi2_meanrev</code> (research) | Buys short dips within an uptrend                                       |
| <code>sol_ema_pullback</code>        | Follows the trend, then waits for a pullback to enter                   |
| <code>vol_breakout</code> (research) | Buys when volatility starts expanding out of a range                    |
| <code>xauby_vwap_pullback</code>     | Follows the trend, referencing volume-weighted average price            |
| <code>donchian_trend</code>          | Buys when price breaks above its recent high/low range                  |
| Research-only short-side group       | Short versions of the strategies above, for testing only                |

★ = the strategy currently trading live (the gold pair)

## 5.2 CDC Action Zone — the Strategy Actually Trading Live

`cdc_action_zone` is the only strategy currently trading live. It runs **stop-and-reverse** style — flipping between long and short as the underlying signal color changes. There's no traditional stop-loss; every exit is driven purely by the signal changing. Position size is calculated as a fixed fraction of capital, and it takes an early one-time partial profit: selling 50% of the position at +12% profit.

| Actual configured value     | Plain-language meaning                                              |
|-----------------------------|---------------------------------------------------------------------|
| Initial protection distance | 2x the market's average volatility                                  |
| Trailing distance           | 1.8x the market's average volatility                                |
| Move stop to breakeven      | Disabled                                                            |
| Capital used per trade      | 95% of allocated capital                                            |
| Early partial profit        | Sell 50% once profit hits 12%                                       |
| Backtest price source       | Uses a different gold-tracking token, since it has a longer history |

## 5.3 What Is the Market Regime Detector

The Market Regime Detector is the part of the system that keeps watch over what kind of market is currently in play — a strong uptrend, a strong downtrend, sideways chop, or a panic sell-off — and tags the current state with a label. That label helps decide how much risk to take, and (if enabled) can automatically pick the strategy best suited to that condition. It analyzes the market from four angles and produces one of 11 possible labels:

### The four angles it analyzes

- **Trend direction** — how short- and long-term moving averages line up, and how far the current price sits from the long-term average
- **Volatility** — whether price is swinging more or less than usual, compared against its own behavior over the last 120 candles (relative, not a fixed number — which is exactly what makes it work across different assets and eras)
- **Liquidity** — trading volume compared to its 20-candle average
- **Macro factors** — combines the dollar index, interest rates, and news sentiment into one score, capping each input so no single one (say, a stray news score) can dominate the result

### All 11 possible labels

| Label                | Meaning                                                                     |
|----------------------|-----------------------------------------------------------------------------|
| BULL_BREAKOUT        | An upside breakout with volume confirmation and expanding volatility        |
| BULL_TREND_STRONG    | A strong uptrend with clear momentum                                        |
| BULL_TREND_WEAK      | A weak uptrend, or macro factors conflicting with the trend                 |
| BEAR_BREAKDOWN       | A downside breakdown with expanding volatility                              |
| BEAR_TREND_STRONG    | A strong downtrend                                                          |
| BEAR_TREND_WEAK      | A weak downtrend                                                            |
| PANIC_SELL           | A panic sell-off — a sharp downtrend with a spike in volatility             |
| LOW_VOL_ACCUMULATION | Low volatility, price hugging the long-term average — an accumulation phase |
| LOW_VOL_RANGE        | Low volatility, but not close enough to the long-term average               |
| VOLATILITY_EXPANSION | Expanding volatility with no clear direction                                |
| SIDeways_CHOP        | Doesn't fit any other condition — directionless chop                        |

### Confidence score

Every label comes with a confidence score, from 5% to 98%. The higher the score, the more confident the system is that the label is correct. It's calculated from several combined factors (trend clarity, how well macro factors align, how clear the volatility reading is, and more) — not a simple yes/no like a basic system would give.

## Protecting against bad calls from too little data

The regime detector needs at least 200 candles of price history to calculate a reliable long-term average. If it's given less than that, it won't dare call the market bullish or bearish — it labels the condition "not yet clear" instead and lowers its confidence score. This protects against a wrong call made from insufficient data.

## 5.4 A Statistical Second Opinion

---

Beyond the main rule-based detector — whose thresholds were set by hand by a person — the system also runs a second checker in parallel that uses pure statistics instead of fixed rules, acting as a **"second opinion"** that carries no bias from anyone's pre-set thresholds.

- It lets the computer try grouping price data on its own, based purely on recent price direction and volatility, without using any fixed rule at all (this method is called a Gaussian Mixture Model, or GMM)
- It only looks at these two things (direction and volatility) because real data proved the system separates volatility far more clearly than direction
- The result is whichever group the latest candle is closest to, along with a confidence level and how long that group has historically tended to last

This result is purely **supporting information** — it never changes the main regime label or the strategy selection. It's off by default; you have to turn it on yourself if you want to see this extra data.

## 5.5 How Accurate Is the Regime Detector, Really

---

A lot of people ask "how accurate is this thing, really?" The answer isn't a percentage-correct score like a school test, because nobody actually knows the "correct" answer for what the market was doing at any given moment. A more meaningful way to measure it is to look at three things instead:

1. **Does it actually separate volatility?** — when the system says the market is in one condition, is the price actually more or less volatile than in a different condition? (This is the main job, since it drives how much risk to take.)
2. **Does a label last long enough to matter?** — if the label changed on every single candle, it would be useless for decision-making.
3. **How accurate is it at predicting direction?** — this is a bonus, not the main measure. It's normal and expected for this to be weak in any market-analysis system.

We tested this using several years of real gold and Bitcoin price history, measuring how clearly the labels separated actual volatility, and how likely that result was to be pure chance.

| What was measured                             | Gold                                          | Bitcoin (comparison test)                     |
|-----------------------------------------------|-----------------------------------------------|-----------------------------------------------|
| How clearly volatility was separated          | <b>Very high confidence</b> this isn't chance | <b>Very high confidence</b> this isn't chance |
| How accurate at predicting the next move      | Very weak (close to a coin flip)              | Weak, but somewhat reliable                   |
| Average candles before a label changes        | About 6-7 candles                             | About 5 candles                               |
| How often the label flips                     | 15% of the time                               | 19% of the time                               |
| Did uptrends actually outperform downtrends   | Yes ✓                                         | Yes ✓                                         |
| Agreement with the statistical second opinion | 50%                                           | 45%                                           |

**The takeaway:** the regime detector does its main job well — it genuinely and reliably separates volatility (not by chance), labels stick around long enough to actually be useful (roughly 5-7 candles on average), and uptrends really do outperform downtrends as expected. Predicting the very next move is weak, which is normal for every market-analysis system out there. **The practical conclusion: use the regime label to decide how much risk to take, not to predict whether price goes up or down next.**

The statistical second opinion agrees with the main detector only about half the time, and that's by design. If the two agreed almost 100% of the time, having a second checker wouldn't add anything. When they disagree, that's a sign the market is genuinely hard to read at that moment — useful information, not a bug in either system.

## Limitations worth knowing

- The volatility-separation result is *small* in absolute terms — most of the market is just noise, and no market-analysis system explains a huge share of it. "Very high confidence" means the separation is real and reliable, not that it's huge.
- The gold price history used for testing comes from a tracking token, since the live gold contract itself has a shorter history.
- The thresholds are still set by hand — they haven't been automatically tuned per asset yet. The statistical second opinion is a first step toward a more data-driven system in the future.
- The return figures measured are descriptive statistics, not a backtest proving any particular strategy is profitable.

## 5.6 Automatically Picking a Strategy for the Market

This feature maps each market condition to the strategy best suited for it. It's opt-in per trading pair, not turned on automatically:

1. The system-wide master switch must be on
2. The switch for that specific pair must also be on

3. If it's a live pair, it also needs explicit operator approval

If a live pair enables this without approval, the bot automatically forces it back to simulation mode. Other behavior: it waits for the market-condition signal to repeat for 3 candles before confirming a real change (to avoid false signals); during a dangerous condition it blocks new trades but keeps existing protection in place; and it requires a stretch of normal, tradeable candles before returning to full trading — with a safe strategy handoff as described in section 3.2.

**Current real status:** the gold pair has this feature turned off — meaning the bot is running CDC Action Zone directly, with no automatic strategy selection involved. The table below shows the mapping configured in the system, ready to use whenever this switch gets turned on in the future:

| Market condition     | Strategy it would pick | Allowed to buy?     |
|----------------------|------------------------|---------------------|
| BULL_BREAKOUT        | cdc_action_zone        | Yes                 |
| BULL_TREND_STRONG    | cdc_action_zone        | Yes                 |
| BULL_TREND_WEAK      | cdc_action_zone        | Yes                 |
| LOW_VOL_ACCUMULATION | bbrsi_mean_reversion   | Yes                 |
| LOW_VOL_RANGE        | bbrsi_mean_reversion   | Yes                 |
| VOLATILITY_EXPANSION | supertrend_ema200      | Yes                 |
| SIDEWAYS_CHOP        | bbrsi_mean_reversion   | Yes                 |
| BEAR_TREND_WEAK      | cdc_action_zone        | Yes                 |
| PANIC_SELL           | None                   | No (dangerous zone) |
| BEAR_BREAKDOWN       | None                   | No (dangerous zone) |
| BEAR_TREND_STRONG    | None                   | No (dangerous zone) |

## 5.7 How Strategy Changes Get Decided With Real Data

The system has a review process before ever swapping out the strategy assigned to a given market condition — a challenger strategy can only replace the current one if it proves it makes more profit across both the data used to tune it and fresh data it has never seen, with enough trades and losses staying within an acceptable range. **The data makes the decision, not a gut feeling.**

**Latest test result:** testing against over 26,000 candles of Bitcoin history, no challenger strategy passed every requirement, so the existing strategy stays in place for every market condition. This test was run on Bitcoin, which isn't currently a live trading pair — it serves as an example of the process for deciding on future strategy changes.

## 6.1 How the Bot Sizes a Position

Every position size is calculated from two things: the total portfolio value right now, and the risk percentage that's configured. In plain terms: as the portfolio grows from profits, the next trade's size automatically grows along with it (compounding) — always capped by a maximum, so no single trade can ever get too large.

**Important:** the risk percentage must be set consistently everywhere in the configuration file, because live trading and backtesting read it from different spots. If they don't match, backtest results stop reflecting live results. The system currently uses 2% everywhere. There's also a startup check that immediately rejects any risk setting above 10%, since that would be dangerously high.

## 6.2 Stop-Loss, Trailing, and Breakeven

Risk-protection mechanisms the system supports (not every strategy uses all of them — see the actual CDC settings in section 5.2):

- **Initial stop-loss** — sets a distance from the entry price based on market volatility
- **Trailing stop** — as price moves favorably, the protection level moves along with it to lock in some profit
- **Breakeven** — once profit hits a set threshold, the stop moves to the entry price exactly, so that trade can no longer lose money (if this is enabled)
- **Multi-check confirmation** — for strategies with no stop-loss directly on the exchange, the signal must repeat across several checks before the bot actually closes the trade, to avoid a false signal from a brief price spike

## 6.3 Taking Partial Profit Early

Configurable in the strategy settings:

```
strategy:
  params:
    cdc_action_zone:
      partial_tp_pct: 12.0
      partial_tp_fraction: 0.5
```

Once an open position's profit reaches the configured percentage from entry, the bot immediately sells the configured fraction, records it as a partial close, then lets the remainder ride on the normal exit signal. This only fires once per trade, and it remembers that it already fired even across a bot restart. For the current gold setup, that means banking 50% profit at +12%, then letting the rest ride until the signal changes. The screen shows this as "pending" or "profit already taken."



## 6.4 The Portfolio-Wide Loss Switch

---

The system has a portfolio-wide emergency switch: if the total portfolio value drops below a set percentage from its highest point ever recorded, the bot immediately blocks new trades (and can be set to force-close open positions too). Think of it as an emergency brake against compounding losses during a prolonged bad stretch.

## 6.5 Short Trading and Its Safety Conditions

---

The current pair can genuinely open short positions with real money, because the connected exchange supports everything that's required. Safety conditions for short trading:

- Must be explicitly allowed for that specific pair in the configuration
- Going live requires an extra switch to be turned on — otherwise it's paper-trading only
- If the connected exchange doesn't support everything required, the system refuses to start at all (choosing safety whenever it isn't sure)
- Leverage defaults to 1x, capped at a maximum of 3x
- If the real-time price feed runs into trouble, the system immediately blocks any new short trades

## 6.6 Keeping Risk Settings Consistent

---

Worth double-checking these values are consistent before going live:

| Setting                                     | Current value for gold |
|---------------------------------------------|------------------------|
| Risk per trade (every location in the file) | 2%                     |
| Maximum position size per trade             | 25%                    |
| Maximum daily loss                          | 6%                     |
| Maximum trades open at once                 | 1                      |
| Maximum leverage                            | 1x                     |

Overall risk limits always apply to every trading pair. Strategy-level risk state is kept separate per pair, but the trading capital itself is intentionally shared.

## 7.1 Where Backtest Data Comes From

Backtesting pulls price data from the global Binance exchange by default, since it has a much longer price history to test against. Live trading uses the actual configured OKX data. Both backtesting and live trading read their settings from the exact same place, so results from both stay consistent with each other.

## 7.2 Backtest Commands

---

```
python scripts/replay_backtest.py --symbol XAUUSD --config bot_config.yaml
python scripts/optimize_pair_configs.py
```

Trying out a new strategy is always a dry run by default — it never overwrites real settings immediately:

```
python scripts/select_pair_strategy.py --symbol BTCUSD --candidates a,b,c
python scripts/select_pair_strategy.py --symbol BTCUSD --candidates a,b,c --apply
```

Only adding `--apply` will actually write a passing winner back into the real configuration.

## 7.3 Regime-Aware Replay

---

This feature can be turned on per pair in the strategy settings — once enabled, the regime detector runs inside the backtest itself. Buy signals get suppressed whenever the market condition at that moment doesn't match the strategy's target conditions, which makes backtest results closer to what actually happens during live trading.

## 7.4 Validating Correctness After a Restart

---

Always run this after a restart, after any incident, or after changing important settings:

```
python scripts/replay_validate.py <run_id> --symbol XAUUSD
```

This checks whether the signals the bot recorded during live trading match what the same strategy would calculate in a backtest — a fast way to catch anything unusual.

## 8.1 The Terminal Dashboard (TUI)

xAuby ships with a terminal dashboard that reads the bot's status in real time — **it never runs the trading loop itself, and it always opens the database in read-only mode**. Closing this screen has no effect on the bot's operation at all.

| Screen            | Shortcut key                    | Shows                                                               |
|-------------------|---------------------------------|---------------------------------------------------------------------|
| Dashboard         | <code>1</code> , <code>d</code> | Chart, pair table, market condition, position, strategy/mode labels |
| Trade log         | <code>2</code> , <code>t</code> | Closed trades and full history                                      |
| Incident explorer | <code>4</code> , <code>i</code> | A timeline of events                                                |

General shortcuts: `[` / `0` previous pair, `]` / `9` next pair, `q` to quit.

Charts and their legends always follow the strategy actually running on the pair you're viewing. The screen adapts automatically to your terminal window's width.

When a position is open and early partial profit-taking is enabled, the screen shows an extra row with the fraction to be banked, the trigger price, and whether it's still pending or already done.

Recommended to run through tmux on a VPS (so the screen survives an SSH disconnect):

```
./scripts/start_dashboard_tmux.sh
./scripts/attach_dashboard_tmux.sh
```

## 8.2 The Read-Only Mobile WebUI

The WebUI is a small, read-only browser dashboard for checking on the bot from a phone or computer — **there is no way to open, close, cancel, or trigger any trade from this screen at all**. It's 100% safe against an accidental tap.

```
./scripts/start_webui.sh
```

The home screen shows the bot's live status, a recent price chart, current strategy signal details, and the partial-profit status of any open position.

### Accessing it from a computer (via SSH tunnel)

```
ssh -L 8787:127.0.0.1:8787 user@your-vps-ip
```

Then open `http://localhost:8787` in your browser.

## Accessing it from your phone

The easiest free option is the Tailscale app: install it on both the VPS and your phone, log into the same account, then reach the WebUI through the VPS's Tailscale address.

Never expose the WebUI directly to the public internet without adding proper authentication first.

## 8.3 Design Philosophy

---

The WebUI is designed to feel like a "private gold desk" for a single owner, checked quickly from a phone or computer — not something you sit down and work in. The one question it must answer within 10 seconds: "Is the bot still running, what's my balance, what position am I holding, has anything happened?"

### Design principles

- **Glanceable first** — balance → current position → signal → everything else, no scrolling needed to see what matters
- **Color only ever means one thing** — green/red/amber always mean profit/loss/warning and nothing else; the brand's gold accent is used only for identity, never for status
- **Read-only means calm** — nothing on screen implies you can control anything from here. Anything abnormal stands out clearly; normal operation stays quiet
- **Mobile-first design** — tap targets sized for fingers; the desktop version gets the same layout centered on the page, not a different one

## 9.1 Setting Up the Telegram Bot

1. Create a bot through [@BotFather](#) on Telegram
2. Find your own Chat ID, for example via [@userinfobot](#)
3. Add the values to your `.env` file:

```
TELEGRAM_ENABLED=true
TELEGRAM_BOT_TOKEN=...
TELEGRAM_CHAT_ID=...
```

## 9.2 All Commands

Only the configured chat can use these — nobody else can command your bot:

| Command              | What it does                                                            |
|----------------------|-------------------------------------------------------------------------|
| <code>/help</code>   | Lists every available command                                           |
| <code>/status</code> | Price, mode, position, and latest signal for every pair                 |
| <code>/pnl</code>    | Portfolio profit/loss over the last 7 and 30 days                       |
| <code>/regime</code> | Latest market condition for each pair                                   |
| <code>/last</code>   | Most recently closed trades                                             |
| <code>/health</code> | Checks whether the bot is running fine, and flags any connection issues |
| <code>/pause</code>  | Temporarily stops new trades, without closing any open position         |
| <code>/resume</code> | Cancels the pause and resumes trading normally                          |

## 9.3 Automatic Alerts

The bot sends these alerts on its own — you don't have to ask for them:

- The bot starting up or shutting down
- A buy or sell going through
- The system blocking a buy due to abnormal market conditions
- A market condition changing
- Protection levels moving
- Daily/weekly summaries (if enabled)

## 9.4 Confirm-Before-Buy Mode and the Emergency Pause

---

Enable confirm-before-buy in the configuration file — once on, the bot sends a confirmation button before opening every trade. If you don't tap it in time, the bot skips that trade entirely.

`/pause` and `/resume` are two-step commands — typing the command only asks for confirmation; you have to tap the confirmation button for it to actually take effect. Pausing blocks new trades across every channel (automatic signals, manual, and confirm-before-buy) without closing anything already open. Critical alerts always come with a button to check status and acknowledge them.

## 10.1 Running Several Bots on One Machine

xAuby separates its configuration from the data it generates while running, which lets you run several bots (different accounts, exchanges, or sets of pairs) side by side on one machine without them colliding — just point each one at its own separate data folder.

```
XAUBY_CONFIG_DIR=instances/acct1 XAUBY_INSTANCE_ID=acct1 \  
python run_xauby.py --live
```

Each instance reads its own `.env` file, so secrets never need to be shared across accounts.

## 10.2 Switching to a Different Exchange

---

You can switch exchanges entirely through the launcher menu, no manual file editing needed — a wizard walks you through picking the exchange, entering your connection details, testing the connection, and offers to restart afterward:

```
exchange:  
  provider: ccxt  
  ccxt_id: kraken  
  api_key_env: KRAKEN_API_KEY  
  api_secret_env: KRAKEN_API_SECRET  
  quote_asset: USDT
```

The system supports adding brand-new exchanges without touching the core code — a developer just needs to add one new connector file, and the system discovers and uses it automatically.

## 10.3 OKX Perpetuals and Short-Trading Safety

---

The certified derivatives path uses isolated-margin perpetual swap contracts. The system refuses to start immediately if the selected exchange doesn't support everything required for short trading (choosing safety whenever it isn't sure), and if the real-time price feed runs into trouble, it immediately blocks any new short trades.

## 11.1 Deploying on a VPS

```
mkdir -p core/logs
env -u DEFAULT_SYMBOL XAUBY_DEFAULT_SYMBOL=XAUUSDTC XAUBY_FOCUS_SYMBOL=XAUUSDTC \
  SIMULATE_ONLY=false BOT_READ_ONLY=false \
  ./venv/bin/python run_xauby.py --live --pair XAUUSDTC

./scripts/start_dashboard_tmux.sh
./scripts/attach_dashboard_tmux.sh
```

Only ever run one instance per data folder — the system already has a lock in place that prevents running two instances against the same folder.

## 11.2 Update and Restart Commands

For routine operations on a VPS once the shortcut commands are installed, use:

```
xauby update  # Pull the latest code from GitHub, then do a checked restart
xauby restart # A checked restart, without pulling new code
```

## 11.3 Diagnosing a Slow VPS

Use this whenever the bot feels sluggish, the screen freezes, or exchange requests keep timing out:

```
./venv/bin/python scripts/vps_latency_check.py
```

| What to check         | What it means                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------|
| Total connection time | Overall time to reach the exchange's API                                                      |
| Time to first byte    | High values usually mean network or API delay                                                 |
| System clock          | Must be synced correctly, since signed requests to the exchange depend on accurate timestamps |
| System load           | Sustained high load beyond available CPU will slow down both the bot and the screen           |
| Disk space            | Low free space or a slow disk affects how the bot writes its data                             |

The dashboard screen also shows timing numbers for each step of the process, which help pinpoint whether a slowdown is coming from the network/exchange side or from local processing.



## 11.4 The Full Safety Checklist

---

- ☐ Reviewed a simulation run before going live
- ☐ The API key has trading permission for the intended account, and **does not** have withdrawal permission
- ☐ All connection details (key/secret/passphrase) are set before restarting into live mode
- ☐ Funds are sitting in the account the bot actually reads for live trading — the screen may show 0.00 if funds are in the wrong wallet
- ☐ Reviewed the risk percentage and overall risk limits for your account size
- ☐ Set the portfolio-wide stop-loss threshold to match your own risk tolerance
- ☐ Only enabled automatic strategy selection after explicit per-pair approval
- ☐ Turned on code-scanning when running a strategy from someone else / not fully trusted
- ☐ Reviewed the results before overwriting any existing strategy configuration
- ☐ Ran the post-restart correctness check successfully
- ☐ Tested Telegram: `/status` shows every pair, `/health` shows nothing unexpected
- ☐ Tested the `/pause` and `/resume` confirmation buttons in simulation mode
- ☐ The bot runs under tmux/systemd, not an unattended SSH window (closing the window would stop it instantly)

## A. Glossary

| Term                        | Plain-language meaning                                                                                                       |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Equity                      | Total account value, including the profit or loss of any open position at the current price                                  |
| Leverage                    | Borrowed money used to increase trade size — the higher it is, the riskier. This system uses 1x, meaning no borrowing at all |
| Long                        | Opening a position betting the price will rise                                                                               |
| Short                       | Opening a position betting the price will fall                                                                               |
| ATR                         | Average True Range — the average size of price movement per candle, used to set protection distances                         |
| EMA                         | A moving average that weights recent data more heavily than older data                                                       |
| Profit Factor (PF)          | Total profit divided by total loss — above 1 means it's profitable overall                                                   |
| Drawdown (DD)               | How far the portfolio value has fallen from its highest point so far                                                         |
| Regime                      | The label the system assigns describing what kind of market condition is currently happening                                 |
| Debounce                    | Waiting for a signal to repeat several times before confirming a real change, to avoid a false signal                        |
| NO_TRADE (dangerous zone)   | A period when the system decides not to open any new trade, while still managing any existing one                            |
| Partial TP                  | Selling part of a position once it hits a set profit target                                                                  |
| Reduce-only                 | An order guaranteed only to shrink a position, never to accidentally open a new one                                          |
| GMM                         | A statistical method that lets a computer group data on its own, without any pre-set fixed rules                             |
| $\eta^2$ (separation score) | A score showing how much a label explains the difference in the data — higher means clearer separation                       |
| p-value                     | A score showing how likely a result is purely due to chance — lower means more trustworthy, less likely to be a coincidence  |

## B. Project Layout

```
xAuby/  
|-- run_xauby.py           # Entry point that starts the engine  
|-- launcher.py           # Interactive menu  
|-- bot_config.yaml       # Main configuration file  
|-- coin_whitelist.json   # List of active trading pairs  
|-- docs/                 # Documentation  
|-- scripts/              # Utility scripts  
|-- tests/                # Test suite  
`-- xauby/                # Core system code  
    |-- launcher/         # Interactive menu  
    |-- engine/           # The core trading engine  
    |-- strategies/       # Strategies and their chart displays  
    |-- runtime/          # Pair list and configuration handling  
    |-- regime/           # The market regime detector  
    |-- backtest/         # Backtesting tools  
    |-- observability/    # Event logging and validation  
    `-- ui/               # The various screens
```

## C. Command Reference

| Command                                                                       | What it does                                              |
|-------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>python -m unittest discover -s tests -q</code>                          | Runs the full test suite                                  |
| <code>python health_check.py</code>                                           | Runs a one-time health check                              |
| <code>python scripts/incident_explorer.py list</code>                         | Lists logged incidents                                    |
| <code>python scripts/replay_validate.py &lt;run_id&gt; --symbol XAUUSD</code> | Validates correctness after a restart                     |
| <code>python scripts/select_pair_strategy.py</code>                           | Trials a strategy change (dry-run / apply)                |
| <code>python scripts/optimize_pair_configs.py</code>                          | Automatically tunes strategy settings, resource-light     |
| <code>python scripts/capture_tui_screenshots.py</code>                        | Regenerates screen previews                               |
| <code>python scripts/fetch_global_klines.py</code>                            | Downloads years of price history for testing              |
| <code>python scripts/regime_forward_validate.py</code>                        | Measures how accurate the regime detector is on real data |
| <code>python scripts/vps_latency_check.py</code>                              | Checks the VPS's network speed                            |

## D. Risk Disclaimer

This software is provided for educational and research purposes only. Trading cryptocurrency and derivatives carries a high risk of losing money. Past performance is never a guarantee of future results. Users must always comply with the terms of the exchange they use and with any applicable laws and regulations in their own

jurisdiction. The authors of this document accept no responsibility for any losses arising from using this software with real funds.

— End of book —